

Niniejsza dokumentacja techniczna zawiera szczegółowy opis architektury, struktury plików, implementacji modułowej oraz formatu składowania danych w desktopowej aplikacji napisanej w języku Python z użyciem biblioteki Tkinter.

Wymagania systemowe i środowiskowe

- **Interpreter języka:** Python w wersji 3.8 lub nowszej.
- **Zależności zewnętrzne:** Brak. Projekt opiera się wyłącznie na bibliotekach standardowych:
 - `tkinter` – odpowiedzialna za renderowanie interfejsu graficznego (GUI).
 - `json` – wykorzystywana do serializacji i deserializacji danych rezerwacji.
 - `os` – używana do operacji na plikach i weryfikacji struktury katalogów.
 - `datetime` – dostarczająca znaczniki czasu (timestamps) do zapisów JSON.

Uwaga dla systemów z rodziny Linux

Biblioteka `tkinter` jest domyślnie dostarczana z instalatorem Pythona dla systemów Windows oraz macOS. W systemach dystrybucji Linux (np. Ubuntu/Debian) może zachodzić potrzeba jej ręcznego doinstalowania za pomocą menedżera pakietów:

```
sudo apt-get install python3-tk
```

Struktura projektu

Repozytorium i struktura katalogów po pierwszym uruchomieniu programu prezentuje się następująco:

```
fluffy-giggle2/
|
├─ Panel-wyboru.py      # Punkt wejścia (Main) – główne okno aplikacji
├─ Podaj_Dane_rez.py    # Logika biznesowa, operacje I/O, okna formularzy
├─ .gitignore
|
├─ rezerwacje/          # Katalog danych (tworzony automatycznie)
  │   └─ rezerwacja_001.json
  │   └─ rezerwacja_002.json
  └─ ...
```

Opis modułów

1. Moduł `Panel-wyboru.py`

Główny punkt wejścia (entry point) do aplikacji. Odpowiada za inicjalizację pętli głównej programu oraz konfigurację nadrzędnego okna klasy `tk.Tk` o stałych wymiarach 1080×720 px (zablokowana możliwość zmiany rozmiaru okna). Komponent ten implementuje:

- Renderowanie menu głównego złożonego z czterech widgetów typu `tk.Button` wraz z obsługą zdarzeń hover.
- Funkcję `otwórz_mape()` – prototypowy widok mapy sali restauracyjnej z 6 stolikami, kolorowanymi statycznie na bazie parzystości numeru (nieparzyste = zielone/wolne, parzyste = czerwone/zajęte). Funkcja ta stanowi makietę interfejsu (mockup) i nie jest jeszcze zintegrowana z bazą plików JSON.

2. Moduł `Podaj_Dane_rez.py`

Rdzeń aplikacyjny zawierający pełną logikę biznesową, obsługę zdarzeń formularzy oraz bezpośredni zapis i odczyt plików. Moduł ten definiuje globalne stałe konfiguracyjne i udostępnia interfejsy formularzy jako osobne podokna.

Funkcje publiczne i API wewnętrzne

`get_next_rezerwacja_number() → int`

Skanuje zawartość katalogu `rezerwacje/` przy użyciu modułu `os`. Analizuje nazwy plików, wyciąga najwyższy dotychczasowy identyfikator liczbowy i zwraca wartość powiększoną o 1. W przypadku braku plików zwraca wartość `1`.

`get_dostepne_stoliki(godzina_str: str) → list[int]`

Zwraca kolekcję identyfikatorów stolików (zakres 1–6), które nie posiadają przypisanej rezerwacji na wskazaną godzinę. Odczytuje sekwencyjnie pliki JSON i porównuje wartość pola `godzina` z parametrem wejściowym `godzina_str` (format `"HH:00"`).

`get_dostepne_godziny(stolik_str: str) → list[str]`

Działa analogicznie do funkcji powyżej, filtrując rezerwacje dla wybranego stolika i zwracając listę wolnych godzin z zakresu otwarcia restauracji, wykluczając godziny już zajęte.

`oblicz_cene(godzina_str: str, uwagi_str: str) → int`

Zwraca wartość całkowitą reprezentującą cenę w PLN. Wyciąga z parametru `godzina_str` wartość numeryczną godziny. Przypisuje cenę bazową (50 PLN dla przedziału 11–20 oraz 100 PLN dla godzin 21–22). Następnie sprawdza czy długość ciągu `uwagi_str` po odrzuceniu białych znaków jest większa od zera — jeśli tak, dolicza 20 PLN.

otworz_okno_rezerwacji()

Inicjalizuje okno formularza (450×600 px). Zawiera triggery walidacyjne (sprawdzenie poprawności imienia, wartości liczbowej gości, unikalności terminu). Po pomyślnej walidacji wywołuje zapis do pliku JSON i zamyka okno komunikatem sukcesu.

otworz_okno_anulowania_rezerwacji()

Generuje okno wyszukiwania (550×400 px). Implementuje mechanizm filtrowania rekordów na podstawie metody `.lower()` na ciągach znaków. Po wybraniu rekordu z poziomu widgetu listy, wywołuje metodę `os.remove()` na skorelowanym pliku JSON.

otworz_widok_dostepnosci_stolikow()

Buduje dynamiczną macierz za pomocą systemu menedżera układu `grid` w Tkinterze (okno 1000×550 px). Mapuje strukturę danych z plików dyskowych do wewnętrznego słownika podręcznego `{stolik: {godzina: imie_nazwisko}}`, co pozwala na błyskawiczne renderowanie statusów komórek tabeli.

Format składowania danych — JSON

Każdy rekord rezerwacji zapisywany jest w osobnym, zdeserializowanym pliku tekstowym według maski nazewnictwa `rezerwacja_NNN.json`, gdzie NNN oznacza numer uzupełniony wiodącymi zerami do trzech pozycji.

Przykład struktury pliku JSON:

```
{
  "numer": 1,
  "imie_nazwisko": "Jan Kowalski",
  "liczba_osob": 4,
  "stolik": 3,
  "godzina": "18:00",
  "uwagi": "alergia na gluten",
  "cena_pln": 70,
  "data_rezerwacji": "2026-05-25T19:30:00.123456"
}
```

Klucz (Pole)	Typ danych	Opis techniczny i semantyka
<code>numer</code>	<code>int</code>	Unikalny, sekwencyjny klucz główny rezerwacji.
<code>imie_nazwisko</code>	<code>str</code>	Ciąg znaków identyfikujący klienta składającego rezerwację.

Klucz (Pole)	Typ danych	Opis techniczny i semantyka
<code>liczba_osob</code>	<code>int</code>	Dodatnia wartość określająca zapotrzebowanie na wielkość stolika.
<code>stolik</code>	<code>int</code>	Identyfikator zasobu restauracyjnego (wartości od 1 do 6).
<code>godzina</code>	<code>str</code>	Klucz czasowy sformatowany według maski <code>HH:00</code> .
<code>uwagi</code>	<code>str</code>	Dowolny ciąg znaków. W przypadku braku uwag - pusty string <code>""</code> .
<code>cena_pln</code>	<code>int</code>	Wyliczony i utrwalony koszt transakcji wyrażony w PLN.
<code>data_rezerwacji</code>	<code>str</code>	Sygnatura czasowa zapisu sformatowana zgodnie z normą ISO 8601.

Konfiguracja stałych globalnych

Parametry konfiguracyjne systemu zostały wyizolowane na początku pliku źródłowego `Podaj_Dane_rez.py`, co pozwala na modyfikację reguł biznesowych bez ingerencji w architekturę kodu:

```

REZERWACJE_DIR    = "rezerwacje"    # Ścieżka relatywna do katalogu z danymi JSON
LICZBA_STOLIKOW   = 6                # Całkowita liczba obsługiwanych stolików
GODZINY_OTWARCIA  = (11, 23)         # Krotka definiująca zakres pracy (od, do -
wyłączając)
CENA_PODSTAWOWA   = 50               # Koszt bazowy w godzinach standardowych (PLN)
CENA_NOCNA        = 100              # Koszt bazowy w godzinach nocnych (PLN)
DODATEK_ZA_UWAGI  = 20               # Opłata dodatkowa za wpis w polu uwag (PLN)

```

Znane ograniczenia i dany dług technologiczny (TODO)

- **Skalowalność I/O:** Przechowywanie danych w płaskich plikach JSON skutkuje złożonością obliczeniową $O(N)$ przy operacjach wyszukiwania i filtrowania. W celach produkcyjnych zalecana jest migracja na relacyjną bazę danych (np. SQLite).
- **Brak wymiaru kalendarzowego:** System obsługuje wyłącznie jeden dzień rozrachunkowy (brak możliwości rezerwacji na inne daty w przód).
- **Prototyp makiety sali:** Metoda `otwórz_mape` w `Panel-wyboru.py` posiada zakodowane na sztywno stany i wymaga pełnego przepisania (oznaczone w kodzie komentarzem `#MIEJSCE NA KOD OSKARA`).

- **Brak funkcji mutacji danych:** System nie pozwala na edycję (UPDATE) rezerwacji. Zmiana parametrów wymusza sekwencję usunięcia i ponownego dodania rekordu.
- **Architektura instancji okien:** Wywoływanie nowych podokien jako instancji `tk.Tk()` zamiast `tk.Toplevel()` powoduje powstawanie izolowanych procesów pętli głównej, co destabilizuje zarządzanie pamięcią podręczną okien. Wskazany refaktoring struktury GUI.